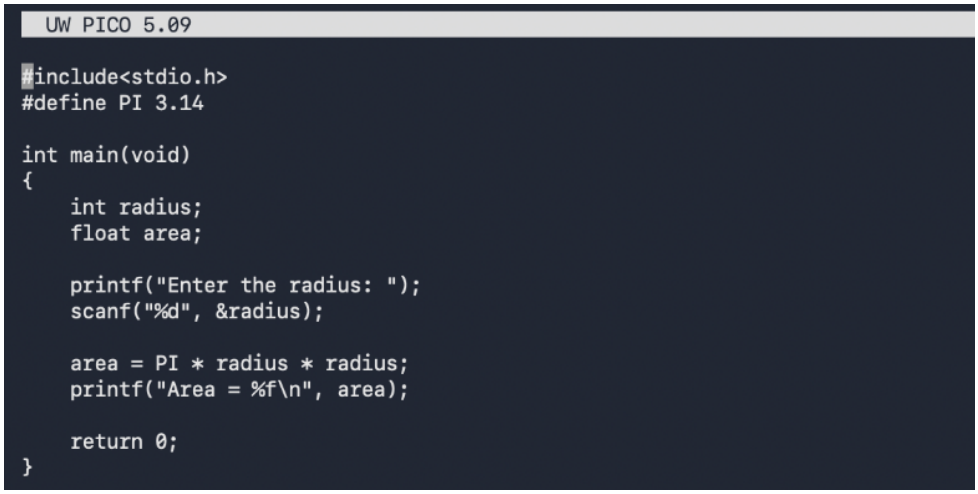
Experiment - 2

Compilation Stages of a C Program

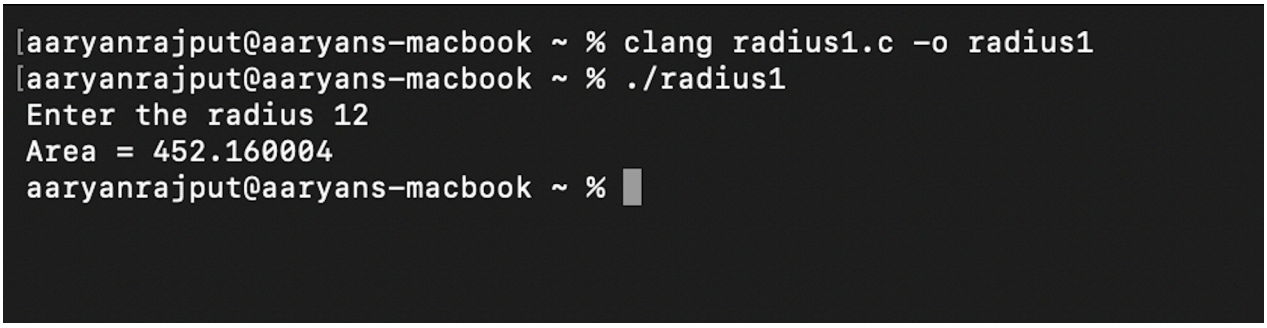
Name: Aaryan Pratap Singh | **SAP ID:** 590032842
Programme: B.Tech CSE | **Year:** 1st Year | **Batch:** 15
Department: Computer Science and Engineering | **Submitted to: DR. Christalin Nelson**
Date: 17 August 2026

1. Writing the C Programs



The file radius.c is open in the Pico editor. It includes the standard input/output header, reads an integer radius, calculates the area using 3.14 x radius x radius, and prints the result.

2. Compiling and Running



The terminal shows the program being compiled and executed. Radius 12 is entered, and the program prints Area = 452.160004, confirming that it runs successfully.

3. Viewing the Preprocessed Program

```
# 472 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/_stdio.h" 3 4
extern const int sys_nerr;
extern const char *const sys_errlist[];

int asprintf(char * *restrict, const char * restrict, ...) __attribute__((__format__ (__printf__, 2, 3)));
char * ctermid_r(char *);
char * fgetln(FILE *, size_t *__len);
const char *fmtcheck(const char *, const char *) __attribute__((format_arg(2)));
int fpurge(FILE *);
void setbuffer(FILE *, char *, int __size);
int setlinebuf(FILE *);
int vasprintf(char * *restrict, const char * restrict, va_list) __attribute__((__format__ (__printf__, 2, 0)));




FILE *funopen(const void *,
     int (* _Nullable)(void *, char *, int __n),
     int (* _Nullable)(void *, const char *, int __n),
     fpos_t (* _Nullable)(void *, fpos_t, int),
     int (* _Nullable)(void *));
# 507 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/_stdio.h" 3 4
# 1 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/secure/_stdio.h" 1 3 4
# 32 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/secure/_stdio.h" 3 4
# 1 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/secure/_common.h" 1 3 4
# 33 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/secure/_stdio.h" 2 3 4




extern int __snprintf_chk (char * restrict , size_t __maxlen, int, size_t,
     const char * restrict, ...);
extern int __vsnprintf_chk (char * restrict , size_t __maxlen, int, size_t,
     const char * restrict, va_list);

extern int __sprintf_chk (char * restrict , int, size_t,
     const char * restrict, ...);
extern int __vsprintf_chk (char * restrict , int, size_t,
     const char * restrict, va_list);
# 508 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/_stdio.h" 2 3 4
# 62 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/stdio.h" 2 3 4
# 2 "radius.c" 2


int main(void)
{
    int radius;
    float area;

    printf("Enter the radius: ");
    scanf("%d", &radius);

    area = 3.14 * radius * radius;
    printf("Area = %f\n", area);

    return 0;
```

This view shows the code after preprocessing. The contents required from stdio.h are inserted, and the macro PI in the calculation is replaced by its value 3.14 before compilation.

4. Viewing the ARM64 Assembly

```
        .cfi_offset w30, -8
        .cfi_offset w29, -16
        mov     w8, #0                          ; =0x0
        str     w8, [sp, #16]                   ; 4-byte Folded Spill
        stur    wzr, [x29, #-4]
        adrp    x0, l_.str@PAGE
        add     x0, x0, l_.str@PAGEOFF
        bl      _printf
        mov     x9, sp
        sub     x8, x29, #8
        str     x8, [x9]
        adrp    x0, l_.str.1@PAGE
        add     x0, x0, l_.str.1@PAGEOFF
        bl      _scanf
        ldur    s1, [x29, #-8]
                                        ; implicit-def: $d0
        mov.16b v0, v1
        sshll.2d        v0, v0, #0
                                        ; kill: def $d0 killed $d0 killed $q0
        scvtf   d1, d0
        mov     x8, #34079                      ; =0x851f
        movk    x8, #20971, lsl #16
        movk    x8, #7864, lsl #32
        movk    x8, #16393, lsl #48
        fmov    d0, x8
        fmul    d0, d0, d1
        ldur    s2, [x29, #-8]
                                        ; implicit-def: $d1
        mov.16b v1, v2
        sshll.2d        v1, v1, #0
                                        ; kill: def $d1 killed $d1 killed $q1
        scvtf   d1, d1
        fmul    d0, d0, d1
        fcvt    s0, d0
        stur    s0, [x29, #-12]
        ldur    s0, [x29, #-12]
        fcvt    d0, s0
        mov     x8, sp
        str     d0, [x8]
        adrp    x0, l_.str.2@PAGE
        add     x0, x0, l_.str.2@PAGEOFF
        bl      _printf
        ldr     w0, [sp, #16]                   ; 4-byte Folded Reload
        ldp     x29, x30, [sp, #32]             ; 16-byte Folded Reload
        add     sp, sp, #48
        ret
        .cfi_endproc
                                        ; -- End function
        .section        __TEXT,__cstring,cstring_literals
l_.str:                                 ; @.str
        .asciz  "Enter the radius: "

l_.str.1:                               ; @.str.1
        .asciz  "%d"

l_.str.2:                               ; @.str.2
        .asciz  "Area = %f\n"
```

Clang has translated the C program into ARM64 assembly for the Mac. These instructions prepare the printf and scanf calls, perform the area calculation, store the result, and return from main.

5. Creating the Object File

```
[aaryanrajput@aaryans-macbook ~ % ls
a.out                   Downloads               Movies                  Public                  radius.i                radius1                 radius1.i               thurs
Desktop                 iCloud Drive (Archive)  Music                   radius.bc               radius.o                radius1.bc              radius1.o               thursday
Documents               Library                 Pictures                radius.c                radius.s                radius1.c               radius1.s
aaryanrajput@aaryans-macbook ~ %
```

The command clang -c radius.c -o radius.o creates machine code without linking. The file command identifies radius.o as a 64-bit Mach-O ARM64 object, confirming successful object-file generation.

6. Result

The terminal views show the complete flow from C source code to execution, preprocessing, ARM64 assembly, and object code. The experiment was completed successfully.